

JavaScript: Comprehensive Interview & Project Guide

Tier 1-4 Coverage (2024-2026)

Contents

Core Concepts

1. Variables, Scope & Hoisting

Variables are containers for values. JavaScript has three declaration keywords:

```
var x = 1;      // Function-scoped, hoisted
let y = 2;      // Block-scoped, Temporal Dead Zone
const z = 3;    // Block-scoped, cannot reassign
```

Key Differences:

- **var**: Function scope, hoisted as undefined
- **let**: Block scope, hoisted but not initialized (TDZ error)
- **const**: Block scope, immutable binding, must initialize

Hoisting Explanation: Declarations are moved to the top of scope before execution. `var` is initialized as undefined. `let/const` enter a Temporal Dead Zone (TDZ) where they cannot be accessed.

Commonly Misunderstood

`const` means the *binding* is immutable, not the object. You cannot reassign, but can mutate object properties: `const obj = {}; obj.x = 1;` is valid.

2. Closures

A closure is a function that “remembers” variables from its lexical scope, even after the outer function has returned.

```
function outer(x) {
  return function inner() {
    return x; // "closes over" x
  };
}
const closure = outer(42);
closure(); // 42
```

Why Closures Matter:

- Data privacy and encapsulation
- Callbacks and event handlers
- Memoization and caching
- Module pattern and namespacing

3. Execution Context & this Binding

this is determined by *how* a function is called (call site), not where it is defined.

```
const obj = {
  name: "Alice",
  greet: function() {
    return "Hello " + this.name;
  }
};

obj.greet(); // "Hello Alice" (obj context)
obj.greet.call({name: "Bob"}); // "Hello Bob" (explicit)
const arrow = () => this; // Inherits parent this
```

Binding Rules (in order):

1. **new binding:** Constructor call creates new this
2. **Explicit binding:** call(), apply(), bind()
3. **Implicit binding:** Object method (receiver is this)
4. **Default binding:** Global object or undefined (strict mode)
5. **Arrow functions:** Lexical (inherit from parent scope)

4. Prototypes & Prototypal Inheritance

Every object has a hidden `[[Prototype]]` property. Objects inherit from other objects, not from classes.

```
function Animal(name) {  
  this.name = name;  
}  
Animal.prototype.speak = function() {  
  return this.name + " makes sound";  
};  
  
const dog = new Animal("Dog");  
dog.speak(); // Inherited from Animal.prototype
```

Prototype Chain:

- Instance object
- Constructor.prototype
- Object.prototype
- null (end of chain)

Property lookup walks up the chain until found or `null` is reached.

5. ES6+ Features

Destructuring

Extract values from objects and arrays.

```
const {name, age} = {name: "Alice", age: 30};  
const [first, second] = [1, 2, 3];  
const {x, ...rest} = {x: 1, y: 2, z: 3};
```

Spread & Rest Operators

```
const arr = [1, 2];  
const spread = [...arr, 3]; // [1, 2, 3]  
  
function rest(a, b, ...args) {  
  console.log(args); // [c, d, e...]  
}  
rest(1, 2, 3, 4, 5);
```

Arrow Functions

Shorter syntax with lexical this and no arguments object.

```
const add = (a, b) => a + b;
const greet = name => `Hello ${name}`;
const obj = () => ({ key: "value" });
```

Template Literals

String interpolation with backticks.

```
const name = "Alice";
const msg = `Hello ${name}!`;
const multiline = `
  Line 1
  Line 2
`;
```

6. Asynchronous Fundamentals

Callbacks

Pass function as argument, executed later.

```
function fetchData(url, callback) {
  setTimeout(() => {
    callback(null, {data: "result"});
  }, 1000);
}
fetchData("/api", (err, data) => {
  if (err) console.error(err);
  else console.log(data);
});
```

Promises

Cleaner async. States: pending → fulfilled or rejected.

```
fetch("/api")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Async/Await

Syntactic sugar over Promises. Looks synchronous, execution is non-blocking.

```
async function getData() {
  try {
    const res = await fetch("/api");
    const data = await res.json();
    return data;
  } catch (err) {
    console.error(err);
  }
}
```

Key Terminology

Term	Definition	Analogy
Scope	Region where variable is accessible	Room in a building
Closure	Function + variables from outer scope	Photo capturing a moment
this	Current execution context	"Self" in a class
Prototype	Object properties shared by instances	Blueprint for objects
Hoisting	Declarations moved to scope top	Items floating to surface
Promise	Placeholder for future value	IOU note
Callback	Function passed as argument	Returning a phone call
Event Loop	Executes callbacks from queue	Restaurant order queue
Microtask	High-priority async task	Express check-out line
Macrotask	Low-priority async task	Regular check-out line
Closure	Function remembers outer scope	Sealed container with contents

How It Works Internally

Event Loop & Execution Model

JavaScript is single-threaded. The event loop manages execution order:

1. Execute all synchronous code (Call Stack)
2. Stack empty? Check Microtask Queue (Promises)
3. Microtask Queue empty? Check Macrotask Queue (setTimeout)
4. Execute one macrotask, return to step 2
5. Microtasks ALWAYS run before macrotasks

Example:

```
console.log('A'); // Sync: immediate
setTimeout(() => console.log('B'), 0); // Macrotask
Promise.resolve().then(() => console.log('C')); // Microtask
console.log('D'); // Sync: immediate
```

Output: A, D, C, B

Execution Context Stack

Each function call creates a new execution context with its own `this`, variables, and scope chain.

```
Global Execution Context
|
+-- outer() Execution Context
    |
    +-- inner() Execution Context
        (stack top, executes first)
```

Prototype Chain Lookup

Property resolution follows the prototype chain:

```
dog.speak()
|-- Check dog object (own properties)
|-- Not found? Check dog.__proto__ (Animal.prototype)
|-- Not found? Check Animal.prototype.__proto__ (Object.prototype)
|-- Not found? Return undefined
|-- Property found? Execute/return it
```

Closure Memory

Closed-over variables are not garbage collected as long as the closure exists.

```
function makeCounter() {
  let count = 0; // Stays in memory (not GC'd)
  return function() {
    return ++count;
  };
}

const counter = makeCounter();
// count variable persists in memory
// even after makeCounter() returns
```

Top 15 Interview Questions

Q1: Explain Closures

Frequency: 95% of interviews

Answer: A closure is a function that remembers variables from its lexical (outer) scope, even after the outer function has executed.

```
function counter() {
  let count = 0;
  return {
    increment: () => ++count,
    decrement: () => --count,
    get: () => count
  };
}

const c = counter();
c.increment(); // 1
c.increment(); // 2
c.get();       // 2
```

Common uses: Data privacy, memoization, callbacks, event handlers, partial application.

Why they ask: Fundamental to modern JavaScript. Tests understanding of scope and memory.

—

Q2: Explain this Binding

Frequency: 90% of interviews

Answer: this is determined by the *call site* (how function is called), not the definition site.

```
const obj = {
  name: "Alice",
  greet: function() {
    console.log(this.name);
  }
};

obj.greet(); // "Alice" (implicit binding)
obj.greet.call({name: "Bob"}); // "Bob" (explicit)
const fn = obj.greet;
fn(); // undefined (default, lost context)
```

Binding hierarchy: new > call/apply/bind > implicit (object) > default (global)

Why they ask: Source of many bugs. Tests scope and context understanding.

—

Q3: var vs. let vs. const

Frequency: 85% of interviews

Aspect	var	let	const
Scope	Function	Block	Block
Hoisting	Yes (undefined)	Yes (TDZ)	Yes (TDZ)
Reassign	Yes	Yes	No
Redeclare	Yes	No	No

Modern Practice: Use const by default, let when reassignment needed, var in legacy code only.

Why they ask: Scope bugs, hoisting issues.

—

Q4: Promises vs. Async/Await

Frequency: 88% of interviews

```
// Promises: chainable
fetch("/api/data")
  .then(res => res.json())
  .then(data => processData(data))
  .catch(err => handleError(err));

// Async/Await: looks synchronous
async function getData() {
  try {
    const res = await fetch("/api/data");
    const data = await res.json();
    return processData(data);
  } catch (err) {
    handleError(err);
  }
}
```

```
}
```

Both are equivalent. Async/await is cleaner for sequential operations, Promises better for parallel operations (`Promise.all()`).

Why they ask: Core async pattern in modern JavaScript.

—

Q5: What is Event Delegation?

Frequency: 75% of interviews

Answer: Attach single listener to parent element, handle events from children via event bubbling.

```
// Inefficient: listener per item
list.querySelectorAll('li').forEach(li => {
  li.addEventListener('click', handleClick);
});

// Efficient: one listener on parent
list.addEventListener('click', (e) => {
  if (e.target.tagName === 'LI') {
    handleClick(e.target);
  }
});
```

Benefits: Performance, handles dynamic items, less memory.

Why they ask: Performance optimization, event handling knowledge.

—

Q6: Explain Prototypal Inheritance

Frequency: 70% of interviews

Answer: Objects inherit directly from other objects via prototype chain. Classes are syntactic sugar.

```
// Constructor + Prototype
function Animal(name) {
  this.name = name;
}
Animal.prototype.speak = function() {
  return this.name + " speaks";
};

function Dog(name) {
  Animal.call(this, name); // Inherit properties
}
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;

// ES6 class syntax (same under the hood)
class Animal { }
class Dog extends Animal { }
```

Why they ask: Tests deep JavaScript understanding.

—

Q7: Explain the Event Loop

Frequency: 80% of interviews

Answer: JavaScript is single-threaded with an event loop managing execution order.

```
console.log('A'); // Sync

setTimeout(() => console.log('B'), 0); // Macrotask

Promise.resolve().then(() => console.log('C')); // Microtask

console.log('D'); // Sync

// Output: A, D, C, B
// Why: Sync first (A, D), then Microtasks (C), then Macrotasks (B)
```

Key insight: Microtasks (Promises, async/await) run BEFORE macrotasks (setTimeout, setInterval).

Why they ask: Critical for async understanding, common gotcha.

—

Q8: Implement Debounce

Frequency: 70% of interviews

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

// Use: search input
const search = debounce((query) => {
  console.log("Searching for:", query);
}, 300);

input.addEventListener('input', (e) => {
  search(e.target.value);
});
```

Use case: Search input, resize handlers, autocomplete.

—

Q9: What is Hoisting?

Frequency: 75% of interviews

Answer: Declarations (variables and functions) are moved to the top of their scope before execution.

```
// var hoisting
console.log(x); // undefined (not error!)
var x = 5;
console.log(x); // 5

// let/const hoisting (TDZ)
console.log(y); // ReferenceError (Temporal Dead Zone)
let y = 5;

// Function hoisting
sayHi(); // Works! Function declarations hoisted
function sayHi() { console.log("Hi"); }
```

TDZ (Temporal Dead Zone): `let/const` are hoisted but not initialized. Accessing them before declaration throws error.

Why they ask: Confuses many developers, causes bugs.

—

Q10: Implement Memoization

Frequency: 65% of interviews

```
function memoize(fn) {
  const cache = {};
  return function(...args) {
    const key = JSON.stringify(args);
    if (key in cache) {
      console.log("From cache");
      return cache[key];
    }
    console.log("Computing");
    return cache[key] = fn(...args);
  };
}

const expensiveFib = memoize((n) => {
  if (n <= 1) return n;
  return expensiveFib(n - 1) + expensiveFib(n - 2);
});

expensiveFib(40); // Fast (cached results)
```

Use case: Expensive computations, API caching.

—

Q11: Explain call, apply, bind

Frequency: 70% of interviews

```
const greet = function(greeting) {
  return greeting + " " + this.name;
};
const person = {name: "Alice"};

// call: immediate, args separate
greet.call(person, "Hi"); // "Hi Alice"

// apply: immediate, args as array
greet.apply(person, ["Hi"]); // "Hi Alice"

// bind: returns new function
const bound = greet.bind(person, "Hi");
bound(); // "Hi Alice"

// Partial application with bind
const add = (a, b) => a + b;
const addFive = add.bind(null, 5);
addFive(3); // 8
```

Why they ask: Fundamental utility functions.

—

Q12: Event Bubbling vs. Capturing

Frequency: 60% of interviews

Answer:

- **Bubbling:** Event travels UP from target → parent → root (default)
- **Capturing:** Event travels DOWN from root → parent → target

```
element.addEventListener('click', handler, false); // Bubble
element.addEventListener('click', handler, true);  // Capture

// Stop propagation
element.addEventListener('click', (e) => {
  e.stopPropagation(); // Stop bubbling up
  e.preventDefault(); // Prevent default action
});
```

Q13: Implement a Simple Promise

Frequency: 55% of interviews

```
class MyPromise {
  constructor(executor) {
    this.state = 'pending';
    this.value = undefined;
    this.reason = undefined;
    this.onFulfilled = [];
    this.onRejected = [];

    executor(this.resolve, this.reject);
  }

  resolve = (value) => {
    if (this.state === 'pending') {
      this.state = 'fulfilled';
      this.value = value;
      this.onFulfilled.forEach(fn => fn(value));
    }
  }

  reject = (reason) => {
    if (this.state === 'pending') {
      this.state = 'rejected';
      this.reason = reason;
      this.onRejected.forEach(fn => fn(reason));
    }
  }

  then = (onFulfill, onReject) => {
    return new MyPromise((resolve, reject) => {
      this.onFulfilled.push((value) => {
        try { resolve(onFulfill(value)); }
        catch(e) { reject(e); }
      });
      this.onRejected.push((reason) => {
        try { reject(onReject(reason)); }
        catch(e) { reject(e); }
      });
    });
  });
}
```

```
}
}
```

—

Q14: Map, Set, WeakMap, WeakSet

Frequency: 55% of interviews

Type	Iterable	Keys	GC
Map	Yes	Any	Strong ref
Set	Yes	N/A	Strong ref
WeakMap	No	Objects only	Weak ref
WeakSet	No	Objects only	Weak ref

```
// Map
const map = new Map();
map.set('key', 'value');
map.get('key'); // 'value'

// Set
const set = new Set([1, 2, 3, 3]); // {1, 2, 3}
set.has(1); // true

// WeakMap (no iteration, keys are weak refs)
const wm = new WeakMap();
const obj = {};
wm.set(obj, 'metadata');
```

Use WeakMap: Private data, DOM metadata, avoiding memory leaks.

—

Q15: Generators & Iterators

Frequency: 50% of interviews

```
// Generator function
function* counter() {
  yield 1;
  yield 2;
  yield 3;
}

const gen = counter();
gen.next(); // {value: 1, done: false}
gen.next(); // {value: 2, done: false}
gen.next(); // {value: 3, done: false}
gen.next(); // {value: undefined, done: true}

// For-of loop
for (const n of counter()) {
  console.log(n); // 1, 2, 3
}

// Custom iterator
const iterable = {
  [Symbol.iterator]() {
    let i = 0;
    return {
      next: () => ({
```

```

        value: i++,
        done: i > 3
      })
    };
  }
};

```

Trade-Offs & Comparisons

Callbacks vs. Promises vs. Async/Await

Aspect	Callbacks	Promises	Async/Await
Readability	Poor	Good	Excellent
Error Handling	Fragile	Try/catch	Try/catch
Chaining	Nested	Clean	Linear
Parallel	Manual	Promise.all()	Promise.all()
Learning Curve	Easy	Moderate	Easy (if know Promises)

Modern Practice: Use async/await for sequential, Promises for parallel operations.

Map vs. Object

Aspect	Object	Map
Key types	Strings/Symbols	Any type
Iteration	Awkward	Built-in (for-of)
Performance	Good	Better for large sets
Size property	Manual count	.size property
Delete	delete key	.delete() method

Use Map for: Key-value pairs with non-string keys, frequent adds/deletes, iteration.

const vs. let

Use const	Use let
By default	When reassignment needed
Prevents accidents	Loops with reassignment
Shows intent (immutable)	Temporary/changing values

Project Applications

Project 1: To-Do App (Beginner)

Concepts: Closures, DOM manipulation, event listeners, array methods.

```

function createToDoApp() {
  let todos = [];
  const list = document.getElementById('list');

  function addTodo(text) {
    todos.push({
      id: Date.now(),
      text: text,
      done: false
    });
  }
}

```

```

    });
    render();
  }

  function toggleTodo(id) {
    const todo = todos.find(t => t.id === id);
    if (todo) todo.done = !todo.done;
    render();
  }

  function deleteTodo(id) {
    todos = todos.filter(t => t.id !== id);
    render();
  }

  function render() {
    list.innerHTML = todos.map(t => `
      <li class="${t.done ? 'done' : ''}">
        ${t.text}
        <button onclick="app.toggle(${t.id})">Done</button>
        <button onclick="app.delete(${t.id})">Delete</button>
      </li>
    `).join('');
  }

  return { addTodo, toggleTodo, deleteTodo };
}

const app = createTodoApp();
app.addTodo("Learn JavaScript");

```

Project 2: Debounce Search (Intermediate)

Concepts: Debounce, async/await, event listeners, closures.

```

function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

const searchHandler = debounce(async (query) => {
  if (!query) return;
  try {
    const res = await fetch(`/api/search?q=${query}`);
    const data = await res.json();
    displayResults(data);
  } catch (err) {
    console.error("Search error:", err);
  }
}, 300);

document.getElementById('search').addEventListener(
  'input',
  (e) => searchHandler(e.target.value)
);

function displayResults(results) {
  const list = document.getElementById('results');

```

```
list.innerHTML = results.map(r => `  
  <div>${r.title}</div>  
`).join('');  
}
```

Project 3: Custom React Hooks (Advanced)

Concepts: React hooks, side effects, closures, async patterns, custom logic.

```
// useFetch Hook  
function useFetch(url) {  
  const [data, setData] = React.useState(null);  
  const [loading, setLoading] = React.useState(true);  
  const [error, setError] = React.useState(null);  
  
  React.useEffect(() => {  
    (async () => {  
      try {  
        const res = await fetch(url);  
        const json = await res.json();  
        setData(json);  
      } catch (e) {  
        setError(e.message);  
      } finally {  
        setLoading(false);  
      }  
    })();  
  }, [url]);  
  
  return { data, loading, error };  
}  
  
// useLocalStorage Hook  
function useLocalStorage(key, initial) {  
  const [state, setState] = React.useState(() => {  
    const stored = localStorage.getItem(key);  
    return stored ? JSON.parse(stored) : initial;  
  });  
  
  React.useEffect(() => {  
    localStorage.setItem(key, JSON.stringify(state));  
  }, [state, key]);  
  
  return [state, setState];  
}  
  
// useDebounce Hook  
function useDebounce(value, delay) {  
  const [debounced, setDebounced] = React.useState(value);  
  
  React.useEffect(() => {  
    const timer = setTimeout(() => setDebounced(value), delay);  
    return () => clearTimeout(timer);  
  }, [value, delay]);  
  
  return debounced;  
}  
  
// Usage  
function SearchComponent() {  
  const [query, setQuery] = React.useState("");
```

```
const debounced = useDebounce(query, 300);
const { data } = useFetch(`/api/search?q=${debounced}`);

return (
  <div>
    <input value={query} onChange={(e) => setQuery(e.target.value)} />
    {data && <div>{data.results.length} results</div>}
  </div>
);
}
```

Common Mistakes & Gotchas

1. Mutating State in React

Wrong

`state.prop = "new"` won't trigger re-render. React doesn't detect mutations.

Correct

`setState({...state, prop: "new"})` creates new object, triggers re-render.

2. Forgetting await on Promises

```
// Wrong: data is a Promise
const data = fetch(url);
console.log(data); // Promise object

// Correct
const data = await fetch(url);
const json = await data.json();
```

3. Closure Variable Capture in Loops

```
// Wrong: i is 3 after loop
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0); // 3, 3, 3
}

// Correct: block scope with let
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0); // 0, 1, 2
}
```

4. this in Callbacks

```
// Wrong: this is window
obj.method = function() {
  setTimeout(function() {
    console.log(this); // window, not obj
  }, 0);
};
```

```
// Correct: arrow function preserves this
obj.method = function() {
  setTimeout(() => console.log(this), 0); // obj
};
```

5. Confusing == and ===

```
// == performs type coercion
0 == false;           // true
"" == false;          // true
null == undefined;    // true

// === strict equality
0 === false;          // false
"" === false;         // false
null === undefined;   // false

// Always use ===
```

6. Not Cleaning Up Event Listeners

```
// Memory leak
element.addEventListener('click', handler);

// Better
element.removeEventListener('click', handler);

// Best in React
React.useEffect(() => {
  element.addEventListener('click', handler);
  return () => element.removeEventListener('click', handler);
}, []);
```

7. Array Mutation in React

```
// Wrong: mutates original
state.arr.splice(0, 1);
state.arr.reverse();

// Correct: creates new array
[...state.arr].slice(1);
[...state.arr].reverse();
```

8. Shallow vs. Deep Copy

```
// Shallow copy (nested objects still referenced)
const copy = {...obj};

// Deep copy (use structuredClone or library)
const deepCopy = structuredClone(obj);
```

9. typeof Array Returns object

```
typeof [1, 2, 3];           // "object" (not "array")
Array.isArray([1, 2, 3]);  // true (use this)
```

10. Missing Return in map

```
// Wrong: returns undefined
arr.map(x => { console.log(x); });

// Correct: explicit return
arr.map(x => { console.log(x); return x * 2; });

// Correct: implicit return
arr.map(x => x * 2);
```

Advanced Patterns

Function Composition

Combine functions to create new functions.

```
const compose = (...fns) => (x) =>
  fns.reduceRight((acc, fn) => fn(acc), x);

const add10 = x => x + 10;
const mul2 = x => x * 2;
const square = x => x * x;

const process = compose(square, mul2, add10);
process(5); // ((5+10)*2)^2 = 1600
```

Currying

Convert multi-argument function to series of single-argument functions.

```
const curry = (fn) => {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn(...args);
    }
    return (...next) => curried(...args, ...next);
  };
};

const add = (a, b, c) => a + b + c;
const curriedAdd = curry(add);
curriedAdd(1)(2)(3); // 6
curriedAdd(1, 2)(3); // 6
```

Module Pattern

Encapsulation using closures.

```
const Calculator = (() => {
  let total = 0;

  return {
    add: (x) => {
      total += x;
      return total;
    },
    reset: () => {
      total = 0;
    },
    getTotal: () => total
  };
})();

Calculator.add(5); // 5
Calculator.add(3); // 8
Calculator.getTotal(); // 8
```

React Hooks Deep Dive

useState

Manage component state with immutability.

```
const [count, setCount] = React.useState(0);

// Update state
setCount(count + 1);

// Functional update (use previous state)
setCount(prev => prev + 1);

// Batching: multiple updates batched in one render
setCount(prev => prev + 1);
setName("Alice");
```

useEffect

Side effects and lifecycle.

```
// Run after every render
React.useEffect(() => {
  console.log("Component rendered");
});

// Run once (mount)
React.useEffect(() => {
  console.log("Mounted");
}, []);

// Run when dependencies change
React.useEffect(() => {
  fetch(url);
}, [url]);

// Cleanup (unmount or before re-run)
React.useEffect(() => {
```

```
const timer = setInterval(() => {}, 1000);
return () => clearInterval(timer);
}, []);
```

useContext

Share state without prop drilling.

```
const ThemeContext = React.createContext();

function Provider() {
  const [theme, setTheme] = React.useState('light');
  return (
    <ThemeContext.Provider value={{theme, setTheme}}>
      <App />
    </ThemeContext.Provider>
  );
}

function Child() {
  const {theme, setTheme} = React.useContext(ThemeContext);
  return <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'light')}>{
    theme}</button>;
}
```

useReducer

Complex state logic.

```
const reducer = (state, action) => {
  switch (action.type) {
    case 'ADD':
      return [...state, action.payload];
    case 'REMOVE':
      return state.filter(i => i.id !== action.id);
    case 'CLEAR':
      return [];
    default:
      return state;
  }
};

function Todo() {
  const [todos, dispatch] = React.useReducer(reducer, []);

  const add = (text) => {
    dispatch({type: 'ADD', payload: {id: Date.now(), text}});
  };

  return <div>...</div>;
}
```

useMemo & useCallback

Performance optimization.

```
// Memoize expensive computation
const expensive = React.useMemo(
  () => compute(data),
```

```
[data] // Only recompute if data changes
);

// Memoize function (prevent child re-renders)
const handleClick = React.useCallback(() => {
  doSomething();
}, [dependencies]);

// Without: new function each render → child re-renders
// With: same function if deps don't change
```

Node.js & Backend Patterns

Module System

```
// CommonJS
module.exports = { add: (a, b) => a + b };
const {add} = require('./math');

// ES6 Modules
export const add = (a, b) => a + b;
export default Math;
import Math, {add} from './math.js';
```

Middleware Pattern

```
// Express
app.use((req, res, next) => {
  console.log("Request:", req.method, req.url);
  next(); // Pass to next middleware
});

// Error middleware (must have 4 params)
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).send("Error");
});

// Custom composition
const compose = (...fns) => (input) =>
  fns.reduce((acc, fn) => fn(acc), input);
```

Error Handling

```
// Try/Catch
try {
  riskyOperation();
} catch (err) {
  console.error("Error:", err.message);
} finally {
  cleanup();
}

// Promise rejection
promise.catch(err => console.error(err));
```

```
// Global handler
process.on('unhandledRejection', (reason) => {
  console.error("Unhandled rejection:", reason);
});
```

One-Page Cheat Sheet

Variables: Use `const` default, `let` for reassign, avoid `var`.

Hoisting: `var` → undefined. `let/const` → TDZ (error if accessed before init).

this binding: Call site determines it. Use arrow for lexical `this`.

Closures: Function + outer scope vars. Enables data privacy, memoization.

Prototypes: Chain: instance → Constructor.prototype → Object.prototype → null.

Async: Callbacks (nesting) → Promises (chainable) → `async/await` (clean).

Event Loop: Sync → Microtasks (Promises) → Macrotasks (`setTimeout`).

Debounce: Delay action until stops. Use: search input.

Throttle: Limit frequency. Use: scroll, resize.

call/apply/bind: Set `this` on function. `call` immediate, `bind` returns function.

Array methods: Pure: `map`, `filter`, `slice`, `concat`. Impure: `splice`, `reverse`, `sort`, `push`.

Object creation: Object literals, constructors, `Object.create`, ES6 classes.

Destructuring: Extract values: `const {x, y} = obj`, `const [a, b] = arr`.

Spread: `[...arr]`, `{...obj}`, `func(...args)`.

React Hooks: `useState` (state), `useEffect` (side effects), `useContext` (share state), `useReducer` (complex logic), `useMemo` (memoize), `useCallback` (memoize function).

Map vs Object: `Map`: any keys, iteration, better performance. `Object`: strings only, simpler syntax.

Memory: Closures keep variables alive. `WeakMap` avoids memory leaks.

Performance: Debounce search input, throttle scroll, memoize expensive functions, lazy load images.

Mistakes: Mutate state, forget `await`, closure capture, `this` in callbacks, `==` vs `===`, memory leaks, shallow copy, forget return in `map`.